

Lab-1-Report

实验目的

- 加深对计算机流水线基本概念的理解；
- 理解MIPS 结构如何用5 段流水线来实现，理解各段的功能和基本操作；
- 加深对数据冲突、结构冲突、控制冲突的理解，并能够分析这些冲突对CPU 性能的影响；
- 进一步理解解决数据冲突的方法，掌握如何应用定性技术来减少数据冲突引起的流水线停顿。

实验平台与实验环境

- Windows 11 虚拟机（QEMU in Arch Linux）
- MIPSsim模拟器

实验步骤与要求

最小实现功能

- 能够模拟MIPS 的5段流水线，以课堂讲授为参考；
- 熟悉图形交互或者命令交互；
- 熟悉单步执行（一次一个时钟周期）、执行到断点、执行到程序结束；
- 掌握流水线各个段、寄存器状态的查看；
- 掌握是否使用定向路径的功能选项；
- 提供程序执行后的性能统计分析；
- 按照MIPS 语法，至少支持 load、store、add、beqz 操作；
- 要执行的程序可以直接输入，或通过文件载入的方式运行

最小实现场景

- 没有任何冲突的流水线场景；
- 有至少一次的RAW 冲突；
- 有至少一次的分支跳转；
- 使用课堂讲授的内容，对上述每一种场景进行分析

实验结果

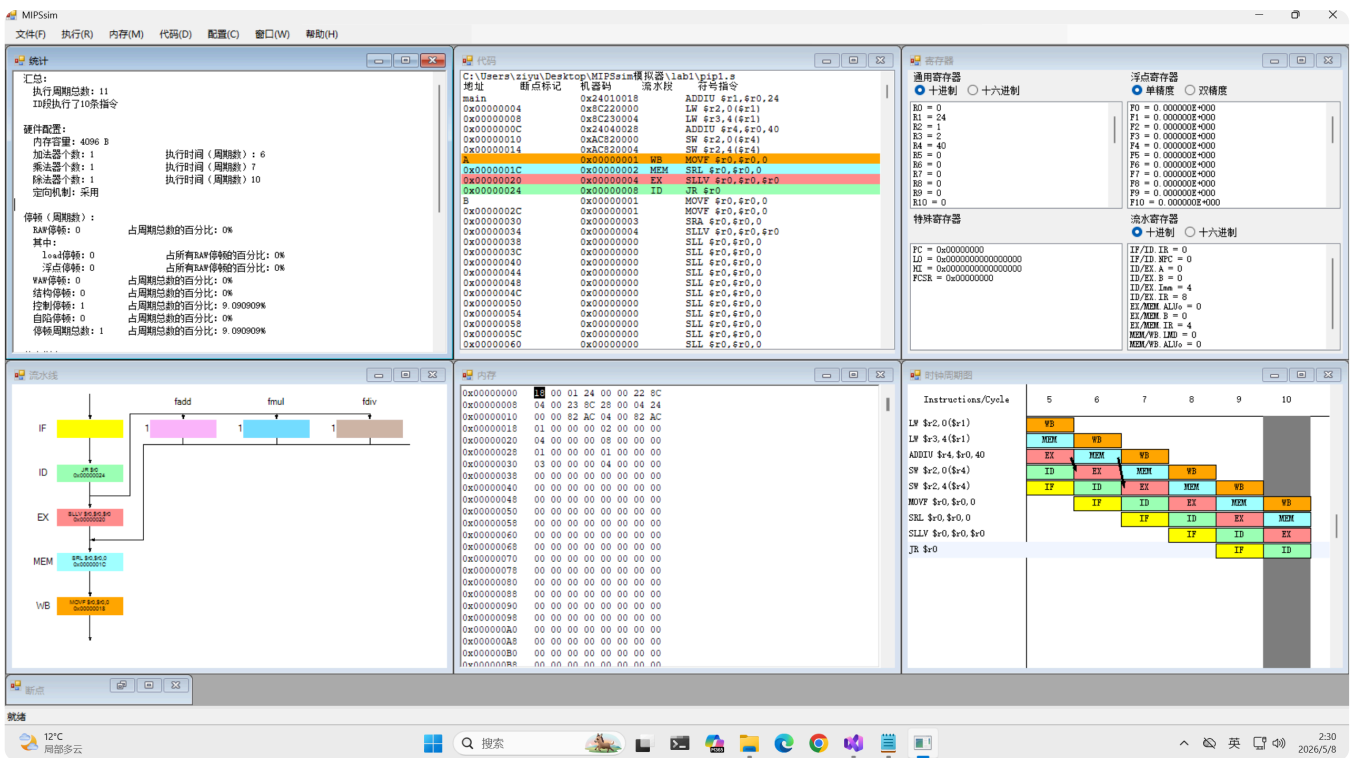
场景1：无冲突的流水线

代码

```
.text
main:
ADDIU $r1,$r0,A
LW $r2,0($r1)
LW $r3,4($r1)
ADDIU $r4,$r0,B
SW $r2,0($r4)
SW $r2,4($r4)
.data
A: .word 1, 2, 4, 8
B: .word 1, 2, 3, 4
```

Unkown

结果



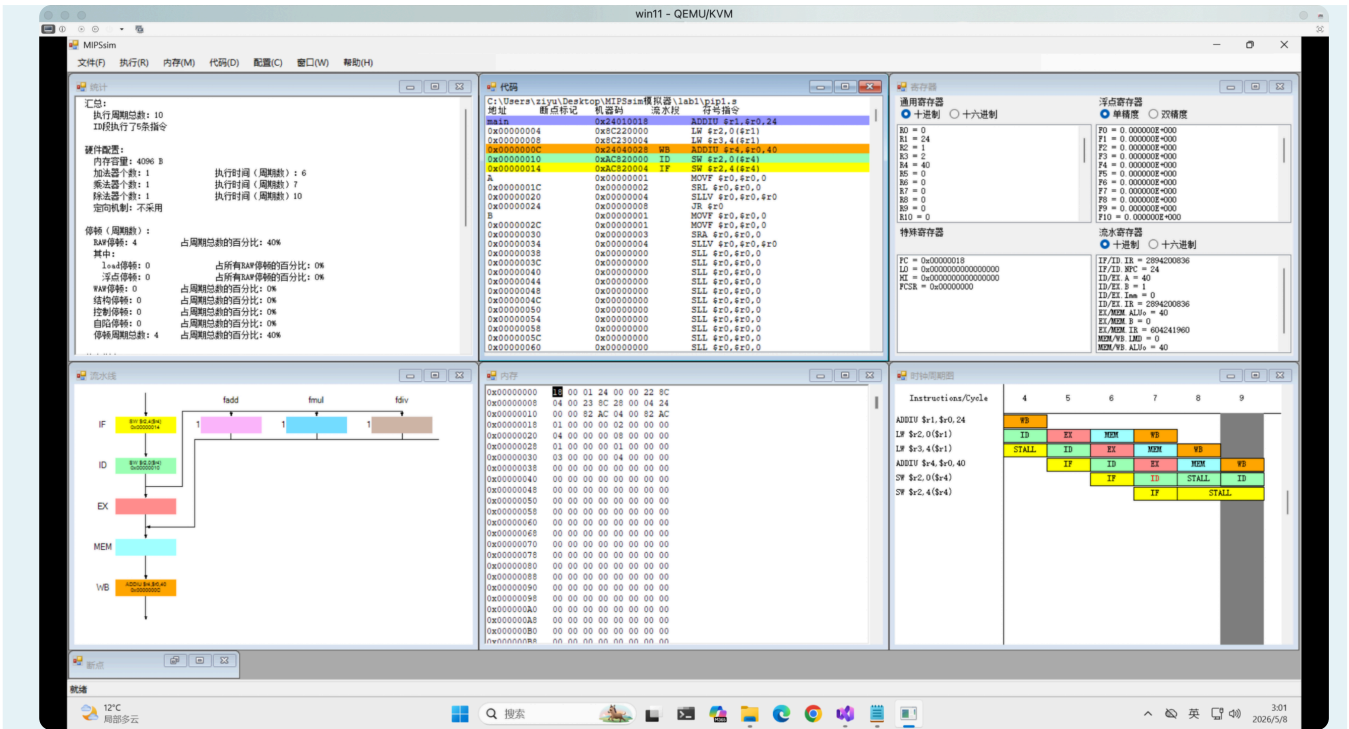
根据 MIPSsim 模拟生成的时钟周期图可以直观地观察到，所有指令在 IF、ID、EX、MEM 和 WB 五个阶段中呈现出阶梯状分布，每一行指令都没有在水平方向上出现被拉长或重复的阶段，也就是说流水线中没有任何停顿（Stall）产生。

原因分析

1. 数据定向解决了计算类冲突。按照传统的流水线逻辑，指令 2 必须等待指令 1 走到 WB 阶段将结果写回寄存器后才能读取。但在开启了定向技术的情况下，指令 1 在 EX 阶段（第 3 周期）计算出结果后，硬件直接通过旁路（Bypassing）将该值传递给正在执行 ID/EX 阶段的指令 2。图中连接 MEM/EX 或 WB/EX 阶段的黑色箭头就是数据在寄存器正式写回前被提前传递并使用的证明。
2. 避开了“加载-使用（Load-Use）”冲突停顿。在 MIPS 流水线中，LW（加载）指令的数据直到 MEM 阶段（第 4 周期）结束时才能从内存取出。指令 2（LW \$r2）加载了 \$r2，但紧随其后的指令 3（LW \$r3）和指令 4（ADDIU \$r4）均未用到 \$r2。直到指令 5（SW \$r2, 0(\$r4)）时才用到了 \$r2。此时指令 2 已经走到了 WB 阶段，数据已经完全就绪。这种指令间距避开了流水线最难以消除的 Load-Use 停顿。
3. 无控制冲突与结构冲突。
 - 控制冲突：代码为纯顺序执行，不包含任何分支跳转指令（如 BEQ、J），因此不存在预测失败导致的流水线冲刷。
 - 结构冲突：MIPSsim 模拟器采用了指令与数据内存分离的哈佛架构（或是理想化的双端口内存），因此 IF（取指）和 MEM（访存）可以同时进行而不会争抢总线。

Note

但是如果不在“配置”中开启“定向”，就会出现 Stall，如图：



因为此时流水线将回归到最基础的执行逻辑，此时数据冲突（Data Hazard）将变得无法避免。导致停顿（Stall）的根本原因在于寄存器读写时序的矛盾：标准的 5 段流水线中，一条指令（如 **ADDIU**）计算出的结果，只有在走完最后一个阶段——写回（WB）时，才会正式写入寄存器堆；但是紧随其后的指令（如 **LW**）为了尽早执行，通常在第二个阶段——译码（ID）时就试图从寄存器堆中读取操作数。

所以，本例中，指令 1 在第 5 个时钟周期才完成写回，指令 2 在第 3 个时钟周期就已经需要读入。此时为了保证程序的逻辑正确，模拟器的硬件逻辑会触发流水线互锁（Interlock）。它会强制“冻结”指令 2 的 ID 阶段，向流水线中插入名为“气泡（Bubble）”的空操作周期（即图中水平延伸的 Stall 块）。指令 2 必须原地踏步，一直等到指令 1 完成 WB 阶段将新值存入寄存器后，才能在下一个周期读取到正确数据。这就是流水线的硬件保护机制。

场景2：RAW冲突

代码

```
.text
main:
ADDIU $r1, $r0, 16
LW $r2, 0($r1)    # 第2步：把内存数据加载到 r2
ADD $r4, $r2, $r3  # 第3步：马上就要用 r2 进行加法计算！
```

Unkown

结果

不开定向

- 不开定向：此时分支策略采取“阻塞取指”。`BEQ` 指令因为要等待 `$r1` 的结果（之前的 `ADDIU` 还没写回），在 ID 阶段卡住了两个周期（黄色 STALL）。由于不开定向，所以第 5、6 周期，`BEQ` 下面那一行是完全空白的，一直等到第8周期 `$r1` 的结果已经确定了，IF 才开始工作。这种策略虽然避免了作废指令，但导致流水线产生了更长的空闲时间，验证了控制冲突对性能的严重影响。
 - 开定向：开启定向后，`BEQ` 指令通过旁路提前获得了 `$r1` 的计算结果，使得控制冲突导致的流水线停顿从 2 个周期降至 1 个周期，取指单元提前 3 个时钟周期获取到了目标指令。整个程序的执行总周期数从 14 降至 11（参考统计窗口），验证了通过硬件优化可以有效缓解控制冲突带来的性能损失。
-

实验感想

通过本次 MIPS 流水线性能分析实验，书本上抽象的 IF、ID、EX、MEM、WB 五段流水线理论转化为了直观的动态感知。在实验过程中，我深刻体会到了数据冲突（RAW）和控制冲突是如何真实地拖慢 CPU 执行效率的，也感受到数据定向的强大用处：通过在硬件中增加旁路，原本需要死等多个周期的停顿被瞬间消除，这种“空间换时间”的设计思路极具工程智慧。